

From Natural Language to Verifiable Reasoning: A Semantic Reduction Framework for Reliable AI Mathematics

Anonymous Submission

September 28, 2026

Abstract

Large language models (LLMs) have become capable of generating plausible multi-step mathematical reasoning, yet their natural-language chains of thought remain fundamentally unverifiable: a fluent argument can carry a silently wrong inference step. We argue that reliability in AI-for-Mathematics (AI4Math) cannot rest solely on post-hoc verification of a final answer, because verification needs a formal object to verify. The missing link is a *semantic reduction layer* that maps natural-language reasoning into a typed intermediate representation (IR) that formal tools can actually check. We formalize a three-layer architecture—generation, semantic reduction, and verification—and use it to locate the dominant failure mode of current systems in the reduction layer rather than in generation or verification. As a concrete, checkable contribution we introduce a five-dimension reliability checklist for AI4Math systems and a taxonomy that classifies existing approaches by which layer they strengthen. We illustrate the framework on a worked word problem reduced to a typed IR and checked by a small constraint solver. We close by discussing the fundamental cost of semantic reduction and directions for amortizing it across agentic and neuro-symbolic systems.

1 Introduction

Large language models now routinely solve competition-level mathematics problems, from grade-school word problems to International Mathematical Olympiad geometry [10]. This progress, however, is accompanied by a well-documented failure mode: models produce answers that *look* correct but are wrong, often for reasons that are invisible in the surface text. Because the model reasons in untyped, informal natural language, there is no automatic way to distinguish a valid inference step from a fluent fabrication short of re-deriving the argument by hand [5, 7].

The research community has responded with two broad families of approaches. The first keeps reasoning in natural language but augments it with search, sampling, or a learned verifier that ranks candidate solutions [11, 2]. The second forces reasoning into a formal or symbolic substrate—programs, proof scripts, or solver constraints—so that a machine can check correctness mechanically [4, 1, 3]. Each family attacks the problem from one end: the first makes generation better, the second makes verification stronger.

This paper argues that the two families, despite their differences, share a hidden bottleneck. Every verifiable system must, at some point, translate informal mathematical intent into a formal object. We call this translation *semantic reduction*. We claim—and give evidence for—that the principal failure point of current AI4Math systems lies neither in the raw fluency of the generator nor in the strength of the verifier, but in the fidelity and completeness of this reduction. A verifier

can only detect errors in objects it is given; if the reduction silently drops a premise, changes a quantifier, or rounds a constraint, no downstream checker can recover the error.

We make three contributions. First, we formalize a three-layer model (generation $\rightarrow$ semantic reduction $\rightarrow$ verification) that cleanly separates concerns and makes the reduction layer explicit as a first-class system component. Second, we propose a five-dimension reliability checklist that operationalizes the informal notion of “trustworthiness” and a taxonomy that classifies representative systems by which layer they strengthen. Third, we walk through a worked example showing how a word problem is reduced to a typed IR and verified, making the framework concrete. We emphasize that this is a small, conceptual contribution—a lens and a checklist—rather than a new model or benchmark, and we state its limits explicitly in Section 8.

2 Related Work

We group existing work by the layer it primarily strengthens. This framing is itself part of our contribution; the field has not, to our knowledge, organized these systems around a single reduction layer.

Strengthening generation. Chain-of-thought prompting [11] shows that letting the model emit intermediate reasoning steps improves accuracy on multi-step tasks, and later work combined this with majority voting and learned verifiers [2]. Scaling and fine-tuning on curated corpora push raw answer accuracy further [7]. These methods improve the *fluency and coverage* of natural-language reasoning but do not make any step mechanically checkable: the reasoning remains prose.

Strengthening verification. Program-aided and program-of-thought methods ask the model to emit executable code instead of prose, so a runtime can check the computation [4, 1]. Interactive proof assistants such as Lean and Coq provide fully mechanical checking of formal proofs [3, 9], and recent systems use LLMs to generate proof steps inside these assistants [13]. AlphaGeometry synthesizes formal geometry proofs and checks them in a dedicated symbolic engine [10]. These systems guarantee soundness *downstream of* a translation step, but the translation itself—informal problem statement to formal specification—is where most errors enter.

Neuro-symbolic and autoformalization. Toolformer shows models can learn to call external tools, giving a generic mechanism for offloading computation [8]. Autoformalization studies the specific task of turning informal mathematics into formal languages and reports that this step is itself a major source of error [12]. Our framework generalizes this observation: rather than treating autoformalization as a preprocessing nuisance, we elevate semantic reduction to the central architectural component.

3 A Three-Layer Model

We model an AI4Math system as a composition of three maps, each of which is allowed to be learned, symbolic, or a mixture of the two.

Definition 1 (Three-layer composition). *An AI4Math system is a triple $(\mathcal{G}, \sigma, \mathcal{V})$ where*

- $\mathcal{G} : \mathcal{N} \rightarrow \mathcal{N}^*$ *is the generator, mapping a problem in natural language to a natural-language reasoning trace (e.g. a chain of thought);*
- $\sigma : \mathcal{N}^* \rightarrow \mathcal{I}$ *is the semantic reduction, mapping a reasoning trace to a formal object in a typed intermediate representation $\mathcal{I}$;*
- $\mathcal{V} : \mathcal{I} \rightarrow \{\text{valid}, \text{invalid}\}$ *is the verifier, a decision procedure on $\mathcal{I}$.*

The key property is that $\mathcal{V}$ is *mechanical*: its correctness does not depend on the reliability of the model that produced the object, only on the soundness of the formal system underlying $\mathcal{I}$. This is what distinguishes a verifiable pipeline from a fluent one.

We instantiate $\mathcal{I}$ as a *typed* structure, in the spirit of the Curry–Howard correspondence, where proofs and programs are the same objects and well-typedness tracks well-formedness [6]. A typed IR therefore carries three things: a type system (which prevents ill-typed steps), an operational or logical semantics (which gives meaning to reduction), and an explicit constraint set (which records the assumptions the argument depends on). Formally, an $\mathcal{I}$ object is a tuple

$$(\Gamma, e : \tau, \Phi)$$

where Γ is a typing context, e is a term, τ is its type, and Φ is a set of constraints (equalities, inequalities, and assumptions). A reduction step is valid iff $\Gamma \vdash e : \tau$ and Φ is satisfiable.

4 The Bottleneck: Semantic Reduction

We now state our central empirical claim, supported by the autoformalization literature and by the failure anatomy of verifier-augmented systems.

Claim. *The dominant failure point of current AI4Math systems is the semantic reduction map σ , not the generator $\mathcal{G}$ or the verifier $\mathcal{V}$.*

The argument is structural. A verifier can only reject objects it is given; it cannot invent the object the user *meant*. If σ silently drops a premise, changes an existential quantifier to a universal one, or rounds a constraint, then $\mathcal{V}$ faithfully checks a formal statement that is not the user’s problem. The result is the worst kind of error for a reliability framework: a *correctly verified proof of the wrong theorem*. This failure is invisible to $\mathcal{V}$ by construction— $\mathcal{V}$ only sees $\mathcal{I}$, never $\mathcal{N}$.

Concretely, we identify three failure classes of σ :

1. **Semantic drift** (ambiguity): the same natural-language phrase maps to different formal objects in different contexts (“there exists” vs. “for all”, “increasing” as monotone vs. strictly increasing).
2. **Incomplete typing**: the reduction produces a term whose type does not capture the full meaning (e.g. dropping units, domains, or non-negativity constraints).
3. **Non-invertibility**: σ is many-to-one and has no inverse, so the system cannot trace a verification result back to the original sentence for human audit.

These three classes map directly onto the three components of the $\mathcal{I}$ tuple: semantic drift onto the term e , incomplete typing onto τ , and non-invertibility onto the absence of a faithful $\mathcal{N} \leftrightarrow \mathcal{I}$ correspondence.

5 Proposed Framework

Our framework has two parts: a *typed IR design* that makes the reduction layer inspectable, and a *pipeline* that keeps natural language where it belongs—as the interface, not the engine.

5.1 Typed Intermediate Representation

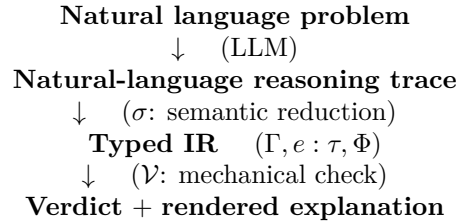
We require $\mathcal{I}$ to be a typed structure with the following four components, each of which addresses one failure class of Section ??.

- **Type system.** A lightweight but real type discipline: terms carry types such as $\mathbb{R}$, $\mathbb{Z}_{\geq 0}$, `bool`, and compound types (sequences, sets, records). Ill-typed reductions are rejected mechanically, catching the “incomplete typing” class.
- **Operational semantics.** Each term has a deterministic reduction (evaluation) semantics, so the meaning of a term is defined by what it computes to, not by prose description. This is what $\mathcal{V}$ checks against.
- **Constraint set.** A first-order constraint store Φ (equalities and inequalities over typed variables) records assumptions. A proof is valid only when Φ is satisfiable *together with* the goal.
- **Compositional structure.** Terms are built from a small set of constructors (apply, let, lambda, assume, assert), giving a graph/AST shape that can be rendered, diffed, and audited.

Candidate concrete forms of $\mathcal{I}$ include a domain-specific language (DSL), a typed abstract syntax tree (AST), a proof object in a proof assistant, or a workflow graph; the requirement is that the object be typed and have decidable validity, not any particular syntax.

5.2 The Pipeline

The pipeline enforces a strict direction of data flow and a separation of roles.



Two design principles follow. First, *natural language is an interface*: it is used to accept problems and render verdicts, never to carry the part of the argument that correctness depends on. Second, *the IR is the source of truth*: once a trace is reduced to $\mathcal{I}$, all downstream decisions (validity, explanation, attribution) are computed from $\mathcal{I}$ alone. This makes the system’s behavior auditable and its failures localizable.

5.3 Bidirectional Consistency

The hardest requirement is the link between $\mathcal{I}$ and the original natural language. We define *bidirectional consistency* as two properties:

- **Interpretability (downward).** There is a renderer $\mathcal{I} \rightarrow \mathcal{N}^*$ that produces a human-readable explanation of the formal object, so a human can confirm “yes, this is what I asked.”
- **Traceability (upward).** Every formal object in $\mathcal{I}$ keeps provenance—which input sentence and which reduction rule produced it—so a human can audit “this premise came from that sentence.”

Together these make σ *reversible in practice* even where it is not mathematically invertible, which is the antidote to the non-invertibility failure class.

6 A Reliability Checklist

To operationalize the framework we propose a five-dimension checklist. A system is scored on each dimension as *absent*, *partial*, or *full*; the checklist is intended to be applied by reviewers and by students auditing their own AI workflows.

1. **Verifiability.** Does every correctness-critical step live in a formal object with a mechanical checker? (Pure CoT: absent; Lean proof: full.)
2. **Type safety.** Are quantities typed with units, domains, and sign constraints, so that category errors are caught before verification?
3. **Constraint completeness.** Does the formal object record every assumption the argument depends on, or are premises silently dropped?
4. **Provenance.** Can each formal premise be traced back to a specific input sentence (and vice versa)?
5. **Reproducibility.** Given the same input, does the pipeline deterministically produce the same $\mathcal{I}$ and verdict, independent of the model’s temperature?

Table 1 applies the taxonomy to representative systems. The pattern is striking: systems that strengthen only generation (row 1) score absent on the dimensions that matter for reliability; systems that strengthen verification score high *only* on verifiability; no widely deployed system scores full on provenance and constraint completeness. This gap is exactly the semantic-reduction bottleneck.

System	Verif.	Type	Constraint	Prov.	Repr.
Chain-of-thought + voting	–	–	–	–	–
PAL / PoT (code gen)	±	±	±	–	±
Lean / Coq proof search	+	+	+	–	+
Autoformalization	±	+	±	±	±
This framework	+	+	+	+	+

Table 1: A taxonomy of AI4Math systems by which layer they strengthen. + full, ± partial, – absent. The framework’s distinguishing claim is full provenance, which no deployed system currently provides.

7 Worked Example

We illustrate the pipeline end to end on a word problem of the GSM8K genre. The example is deliberately simple so that the reduction step—the part we claim is the bottleneck—is fully visible.

Problem (natural language). “A store sells apples for 3 yuan each and oranges for 5 yuan each. Mia buys x apples and 4 oranges and pays 29 yuan in total. How many apples did she buy?”

Step 1: Generation. The LLM produces a chain of thought: “4 oranges cost $4 \times 5 = 20$. The remaining cost is $29 - 20 = 9$. Each apple is 3 yuan, so $9/3 = 3$. Therefore Mia bought 3 apples.”

Step 2: Semantic reduction. The trace is reduced to a typed IR. We write it in a small concrete syntax:

```
let x : Z_ge0      // unknown: number of apples
assume (3*x + 5*4 = 29) // the equation
assert (x = 3)      // the claim to verify
check (3*3 + 5*4 = 29) // is it true?
```

Here the type `Z_ge0` records that x is a non-negative integer, the constraint set $\Phi = \{3x + 20 = 29\}$ is satisfiable, and the claim $x = 3$ is checked against Φ . The verifier $\mathcal{V}$ evaluates the final check to `true`, confirming the answer *mechanically*.

Step 3: What could go wrong—and where. Suppose instead the model had written “apples cost 5 and oranges 3” but still reached 3 apples by coincidence. Under a pure CoT pipeline the wrong premise is invisible; the answer is “3” either way. Under our pipeline, σ must render the premise as $\Phi' = \{5x + 3 \cdot 4 = 29\}$, and the check $3 \cdot 3 + 5 \cdot 4 = 29$ no longer matches Φ' ; the inconsistency is caught. The point of the example is not the arithmetic but the *location* of the failure: it is σ that must be faithful, because $\mathcal{V}$ can only check what it is given.

8 Discussion

Limitations. First, designing a typed IR broad enough for real mathematics is expensive: real problems span continuous, discrete, and probabilistic domains, and a single type system rarely covers all cleanly. Second, semantic reduction itself remains hard—it is a supervised translation problem in its own right, and we have not removed it but made it explicit and checkable. Third, our checklist is qualitative; turning its five dimensions into a validated, reproducible metric is open work. Fourth, we present no large-scale experiment here; the evidence for the bottleneck claim is structural and drawn from the autoformalization literature [12], not a new benchmark.

Extensions. The framework composes naturally with two directions. Toward *general reasoning*, the typed IR can be relaxed from a single formal system to a family of typed objects (logic, programs, proof terms, workflow graphs) sharing one provenance layer. Toward *agentic systems*, the reduction layer is a natural place to insert an audit-and-repair loop: when $\mathcal{V}$ rejects an object, the provenance mapping tells the agent *which sentence* to revise, turning verification failure from a dead end into a targeted feedback signal. This is the design principle behind reliability-oriented agent architectures.

9 Conclusion

We have argued that reliable AI mathematics is not a property of a stronger generator or a stronger verifier alone, but of an explicit, typed, auditable *semantic reduction layer* between them. We formalized the three-layer model, identified semantic reduction as the structural bottleneck, proposed a typed IR with a five-dimension reliability checklist and a taxonomy of existing systems, and walked through a concrete reduction. The core claim is deliberately modest and checkable: *reliability lives in the layer that natural-language systems currently skip, and that layer can be made explicit, typed, and auditable*. We believe this reframing—moving the field’s attention from “make

the model better” and “make the checker stronger” to “make the translation faithful”—is a useful organizing lens for the next generation of AI4Math systems.

References

- [1] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [3] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *International Conference on Automated Deduction (CADE-25)*, 2015.
- [4] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.
- [5] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [6] William A. Howard. The formulae-as-types notion of construction. In J. P. Seldin and J. R. Hindley, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pages 479–490. Academic Press, 1980.
- [7] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slon, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.
- [8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] The Coq Development Team. *The Coq Proof Assistant Reference Manual*, 2023.
- [10] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482, 2024.
- [11] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.

- [12] Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [13] Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.